

AssanPay — Third-Party API Request / Response

Core-banking & external service payloads for IFT, IBFT, Onboarding endpoints | Generated: 23 Sep 2026

POST JSON

URL: GetURL("11") — DB config

Client: PostAsync<T>

Login: real flow (SP-validated)

AccountList / Unikrew: mocked

Contents

- 1. IFT Payment — IftPaymentRequestEntity / IftPaymentResponseEntity
- 2. IBFT Payment — IbftPaymentRequestEntity / IbftPaymentResponseEntity
- 3. IFT Title Fetch — IftTitleFetchRequestEntity / IftTitleFetchResponseEntity
- 4. IBFT Title Fetch — IbftTitleFetchRequestEntity / IbftTitleFetchResponseEntity
- 5. Balance Inquiry — BalanceInquiryRequestEntity / BalanceInquiryResponseEntity
- 6. Onboarding: Login — LoginRequestEntity / LoginResponseEntity
- 7. Onboarding: Register — RegisterRequestEntity / RegisterResponseEntity
- 8. Onboarding: Account List (mocked) — AccountListRequestEntity / AccountListResponseEntity
- 9. Onboarding: Unikrew Upload (mocked) — UploadDocumentUnikrewRequestEntity / UploadDocumentUnikrewResponseEntity
- 10. Onboarding: NADRA Verify Fingers — VerifyFingerRequestEntity / VerifyFingerResponseEntity
- 11. Onboarding: Face Liveliness — VerifyLivelinessRequestEntity / VerifyLivelinessResponseEntity
- 12. Onboarding: Unikrew CNIC Document Validation — ValidateDocumentUnikrewRequestEntity / ValidateDocumentUnikrewResponseEntity
- 13. Onboarding: Customer Detail — CustomerDetailRequestEntity / CustomerDetailResponseEntity
- 14. Inquiry: Accounts by CNIC — CnicTitleFetchRequestEntity / CnicTitleFetchResponseEntity

Common envelope fields: All core-banking requests carry a header block — strAppID, strADC_TranCode, strDelimiter, strDate, strTime, strNetID, intRefNum, intSTAN, strTranMsg, strMerName, strMCC, strToken — plus device fields (Client_IP, Device_IP, Device_version, Device_Type, Device_MEI, Application_Version, UUID, Cusomter_ID, Latitude, Langitude, Channel_type, transmissionDateTime). Responses carry Response_Code / Response_Desc (00 = success).

1. IFT Payment

Used by: Payment/IFT, RAAST/Ibft

Repo: FundsTransferRepository.cs:32

Entity: IftPaymentApi.cs

→ Request — IftPaymentRequestEntity

```
{
  "strAppID": "", "strADC_TranCode": "", "strDelimiter": "",
  "strDate": "20260923", "strTime": "143000", "strNetID": "",
  "intRefNum": "1234", "intSTAN": "12345678",
  "strTranMsg": "", "strMerName": "", "strMCC": "", "strToken": "",
  "Client_IP": "", "Device_IP": "", "Device_version": "", "Device_Type": "",
  "Device_MEI": "", "Application_Version": "", "UUID": "123456",
  "Cusomter_ID": "", "Latitude": "", "Langitude": "", "Channel_type": "",
  "transmissionDateTime": "2026-09-23 02:30:00.000",
  "agentAccount": "", "amount": "1000", "channelId": "",
  "fromAccount": "", "password": "", "processingCode": "",
  "stan": "123456", "toAccount": "", "accountTitle": "",
  "fromAccountTitle": "", "userId": "",
  "Description": "transfer", "PurposeOfPayment": "", "CNIC": ""
}
```

← Response — IftPaymentResponseEntity

```
{
  "return": {
    "agentAccount": "", "amount": "", "channelId": "", "fromAccount": "",
    "password": "", "processingCode": "", "stan": "",
    "status": "00", "statusDescription": "Success",
    "toAccount": "", "userId": "", "PurposeOfPayment": "",
    "TransmissionDateTime": ""
  },
  "Response_Code": "00",
  "Response_Desc": "Success"
}
```

2. IBFT Payment

Used by: Inquiry/IBFT, Inquiry/IBFT_V1

Repo: FundsTransferRepository.cs:64 (commented/mock)

Entity: IbftPaymentApi.cs

→ Request — IbftPaymentRequestEntity

```
{
  "strAppID": "", "strADC_Trancode": "", "strDelimiter": "",
  "strDate": "", "strTime": "", "strNetID": "",
  "intRefNum": "", "intSTAN": "", "strTranMsg": "",
  "strMerName": "", "strMCC": "", "strToken": "",
  "Client_IP": "", "Device_IP": "", "Device_version": "", "Device_Type": "",
  "Device_MEI": "", "Application_Version": "", "UUID": "",
  "Cusomter_ID": "", "Latitude": "", "Langitude": "", "Channel_type": "",
  "transmissionDateTime": "",
  "accountTitles": "", "agentAccount": "", "amount": "1000",
  "branchCode": "", "channelId": "", "dateAndTime": "",
  "fromAccount": "", "fromAccountTitle": "", "fromIBAN": "",
  "password": "", "processingCode": "", "recordData": "",
  "stan": "", "status": "", "statusDescription": "",
  "toAccount": "", "toIBAN": "", "toIMD": "",
  "transactionID": "", "userId": "", "webService": "",
  "Description": "", "CNIC": ""
}
```

← Response — IbftPaymentResponseEntity (flat, no nested return)

```
{
  "Response_Code": "00",
  "Response_Desc": "Success"
}
```

3. IFT Title Fetch

Used by: Inquiry/IFT

Repo: InquiryRepository.cs:142

Entity: IftTitleFetchApi.cs

→ Request — IftTitleFetchRequestEntity

```
{
  "strAppID": "", "strADC_Trancode": "", "strDelimiter": "",
  "strDate": "", "strTime": "", "strNetID": "",
  "intRefNum": "", "intSTAN": "", "strTranMsg": "",
  "strMerName": "", "strMCC": "", "strToken": "",
  "Client_IP": "", "Device_IP": "", "Device_version": "", "Device_Type": "",
  "Device_MEI": "", "Application_Version": "", "UUID": "",
  "Cusomter_ID": "", "Latitude": "", "Langitude": "", "Channel_type": "",
  "transmissionDateTime": "",
  "agentAccount": "", "amount": "", "channelId": "",
  "Account": "0345001234567", "password": "", "processingCode": "",
  "stan": "", "userId": "", "webService": "",
  "id_type": "", "id_value": "", "CNIC": ""
}
```

← Response — IftTitleFetchResponseEntity

```
{
  "return": {
    "accountTitle": "ALI KHAN", "account": "0345001234567",
    "agentAccount": "", "channelId": "", "fromAccount": "",
    "password": "", "processingCode": "", "stan": "",
    "status": "00", "statusDescription": "Success",
    "userId": "", "TransmissionDateTime": "",
    "idType": "", "idValue": ""
  },
  "Response_Code": "00",
  "Response_Desc": "Success"
}
```

4. IBFT Title Fetch

Used by: Inquiry/IBFT

Repo: InquiryRepository.cs:174 (commented)

Entity: IbftTitleFetchApi.cs

→ Request — IbftTitleFetchRequestEntity

```
{
  "strAppID": "", "strADC_TranCode": "", "strDelimiter": "",
  "strDate": "", "strTime": "", "strNetID": "",
  "intRefNum": "", "intSTAN": "", "strTranMsg": "",
  "strMerName": "", "strMCC": "", "strToken": "",
  "Client_IP": "", "Device_IP": "", "Device_version": "", "Device_Type": "",
  "Device_MEI": "", "Application_Version": "", "UUID": "",
  "Cusomter_ID": "", "Latitude": "", "Langitude": "", "Channel_type": "",
  "transmissionDateTime": "",
  "accountTitle": "", "agentAccount": "", "amount": "", "branchCode": "",
  "channelId": "", "dateAndTime": "", "fromAccount": "", "fromIBAN": "",
  "password": "", "processingCode": "", "recordData": "",
  "stan": "", "status": "", "statusDescription": "",
  "toAccount": "", "toIBAN": "", "toIMD": "",
  "transactionID": "", "userId": "", "webService": "", "CNIC": ""
}
```

← Response — IbftTitleFetchResponseEntity (double-nested)

```
{
  "IBFTTitleFetchResponse": {
    "return": {
      "accountTitle": "ALI KHAN", "agentAccount": "", "amount": "",
      "dateAndTime": "", "fromAccount": "", "password": "",
      "processingCode": "", "recordData": "", "stan": "",
      "toAccount": "", "toIBAN": "", "toIMD": "",
      "transactionID": "", "userId": "", "webService": ""
    }
  },
  "Response_Code": "00",
  "Response_Desc": "Success"
}
```

5. Balance Inquiry

Used by: Inquiry/Balance (mocked in service)

Repo: InquiryRepository.cs:37

Entity: BalanceInquiryApi.cs

→ Request — BalanceInquiryRequestEntity

```
{
  "strAppID": "", "strADC_TranCode": "", "strDelimiter": "",
  "strDate": "", "strTime": "", "strNetID": "",
  "intRefNum": "", "intSTAN": "", "strTranMsg": "",
  "strMerName": "", "strMCC": "", "strToken": "",
  "Client_IP": "", "Device_IP": "", "Device_version": "", "Device_Type": "",
  "Device_MEI": "", "Application_Version": "", "UUID": "",
  "Cusomter_ID": "", "Latitude": "", "Longitude": "", "Channel_type": "",
  "transmissionDateTime": "",
  "agentAccount": "", "branchCode": "", "channelId": "",
  "fromAccount": "0345001234567", "password": "", "processingCode": "",
  "stan": "", "userId": "", "webService": "", "CNIC": ""
}
```

← Response — BalanceInquiryResponseEntity

```
{
  "Return": {
    "Amount": "150000.00",
    "BranchCode": "001",
    "FromAccount": "0345001234567",
    "CURRENCY_CODE": "PKR"
  },
  "Response_Code": "00",
  "Response_Desc": "Success"
}
```

6. Onboarding — Login

Used by: Account/Login

Repo: AccountRepository.cs:170 — real SP flow (external call mocked at repo)

Entity: LoginAPI.cs

→ Request — LoginRequestEntity

```
{
  "strAppID": "", "strADC_TranCode": "", "strDelimiter": "",
  "strDate": "", "strTime": "", "strNetID": "",
  "intRefNum": "", "intSTAN": "", "strTranMsg": "",
  "strMerName": "", "strMCC": "", "strToken": "",
  "UserName": "ali.khan", "CLIENT_IP": "", "deviceIp": "",
  "deviceVersion": "", "deviceType": "", "CNIC": "4210112345678"
}
```

← Response — LoginResponseEntity

```
{
  "Response_Code": "00",
  "Response_Desc": "Success",
  "strToken": "<core-banking-token>",
  "UserName": "ali.khan"
}
```

Note: External core-banking call is mocked at repository layer (returns hardcoded "00"). Credential validation runs through stored procedures; JWT BearerToken is generated locally by AuthTokenService.

7. Onboarding — Register

Used by: Account/Register

Repo: AccountRepository.cs:213

Entity: RegisterAPI.cs

→ Request — RegisterRequestEntity

```
{
  "strAppID": "", "strADC_Trancode": "", "strDelimiter": "",
  "strDate": "", "strTime": "", "strNetID": "",
  "intRefNum": "", "intSTAN": "", "strTranMsg": "",
  "strMerName": "", "strMCC": "", "strToken": "",
  "CNIC": "4210112345678", "AccountNo": "0345001234567",
  "MobileNo": "03001234567", "UserName": "ali.khan",
  "CLIENT_IP": "", "deviceIp": "", "deviceVersion": "", "deviceType": ""
}
```

← Response — RegisterResponseEntity

```
{
  "Response_Code": "00",
  "Response_Desc": "Success"
}
```

8. Onboarding — Account List (MOCKED)

Mocked in AccountService.cs:135 + AccountRepository

Entity: AccountListApi.cs

→ Request — AccountListRequestEntity

```
{
  "strAppID": "MB", "strADC_Trancode": "258", "strDelimiter": "^",
  "strDate": "092026", "strTime": "02:30:00 PM", "strNetID": "test",
  "intRefNum": "327010", "intSTAN": "12345678",
  "strTranMsg": "REGISTRATION", "strMerName": "Branch", "strMCC": "MCC",
  "strToken": "", "ClientIP": "", "DeviceIP": "", "Deviceversion": "",
  "DeviceType": "", "DeviceMEI": "", "ApplicationVersion": "",
  "UUID": "", "CustomerID": "1234", "Latitude": "", "Longitude": "",
  "Channeltype": "channeptype", "transmissionDateTime": "",
  "Request": { "CNIC": "4210112345678" }
}
```

← Response — AccountListResponseEntity (mock returns 2 accounts)

```
{
  "Response": {
    "CASA_Account_List": [
      {
        "Account_Number": "0345001234567",
        "Account_IBAN": "PK36MEZN0001230012345678",
        "Account_Title": "ALI KHAN",
        "Account_Type": "CURRENT",
        "Account_Currency": "PKR",
        "Account_Status": "ACTIVE",
        "Account_Description": "",
        "ACCT_STATUS_CODE": "A"
      }
    ]
  },
  "Response_Code": "00",
  "Response_Desc": "Success"
}
```

9. Onboarding — Unikrew Document Upload (MOCKED)

Mocked in AccountService.cs — CNIC parity logic

Repo: AccountRepository.cs:82

Entity: UploadDocumentUnikrewAPI.cs

→ Request — UploadDocumentUnikrewRequestEntity

```
{
  "comparisonRequired": true,
  "sameNicRequired": true,
  "ocrRequired": true,
  "livenessImage": "<base64>",
  "nicFrontImage": "<base64>",
  "nicBackImage": "<base64>",
  "userName": "",
  "password": ""
}
```

← Response — UploadDocumentUnikrewResponseEntity (even-CNIC → approve)

```
{
  "isSmartNic": true,
  "ocrResponse": {
    "uniqueIdentifier": "4210112345678",
    "fields": [
      { "name": "name", "value": "ALI KHAN" },
      { "name": "cnic", "value": "4210112345678" },
      { "name": "dob", "value": "1990-01-01" }
    ],
    "rawData": "",
    "transliteratedText": null,
    "isNicSame": true
  },
  "faceComparisonResponse": {
    "thresholds": { "low": 0.30, "mid": 0.60, "high": 0.80 },
    "confidence": 0.95
  }
}
```

Mock logic: CNIC last digit even → confidence 0.95 (> high 0.80 → auto-approved, step_id=4, status_id=2). Odd → confidence 0.45 (mid zone → manual review, step_id=5, status_id=1). Stored via SP usp_bankislami_upload_customer_document_unikrew.

10. Onboarding — NADRA Verify Fingers (fingerprint biometric)

Used by: Account/VerifyFingers, Account/VerifyFingersForAccountSelection

Repo: AccountRepository.cs:13 → GetURL("VerifyFinger") — LIVE (not mocked)

Entity: VerifyFingerPrintRequestApi.cs

NADRA call: This is the fingerprint verification call to NADRA (proxied via the bank's VerifyFinger URL). Confirmed by comment at AccountService.cs:1412 — "No reponse from nadra." Status codes handled: 100 = success, 119 = system error, 122 = "fingerprints does not matched", 101 = CNIC not found. **Not mocked** — hits real NADRA endpoint.

→ Request — VerifyFingerRequestEntity

```
{
  "strAppId": "MB", "strADC_TransCode": "", "strDelimiter": "",
  "strDate": "092026", "strTime": "02:30:00 PM", "strNetID": "",
  "intRefNum": "123456789", "intSTAN": "12345678",
  "strTransMsg": "", "strMerName": "", "strMCC": "", "strToken": "",
  "ClientIP": "", "DeviceIP": "", "DeviceVersion": "", "DeviceType": "",
  "DeviceMEI": "", "ApplicationVersion": "", "UUID": "",
  "CustomerID": "1234", "Latitude": "", "Longitude": "",
  "ChannelType": "channeptype", "transmissionDateTime": "",
  "citizenNumber": "4210112345678",
  "contactNumber": "03001234567",
  "fingerIndex": "1",
  "fingerTemplate": "<base64-fingerprint-template>",
  "areaName": "SINDH",
  "accountType": "",
  "R1": "?", "R2": "?", "R3": "?", "R4": "?",
  "CNIC": "4210112345678",
  "sessionId": "",
  "transactionId": ""
}
```

← Response — VerifyFingerResponseEntity

```
{
  "Status_Code": "100",
  "Message": "successful",
  "Session_ID": "3712100001813731502",
  "Citizen_Number": "4210112345678",
  "FINGER_ONE": "M",
  "FINGER_TWO": "M",
  "FINGER_THREE": "M",
  "FINGER_FOUR": "M",
  "Response_Code": "00",
  "Response_Desc": "Success"
}
```

Status codes: 100 = success · 119 = system error · 122 = fingerprint mismatch (returns bilingual message) · 101 = CNIC not found. Area name is derived from the first CNIC digit via SP usp_account_get_province_list.

11. Onboarding — Face Liveliness Check

Used by: Account/VerifyLiveliness

Repo: AccountRepository.cs:40 → GetURL("Liveliness") — LIVE

Entity: VerifyLivelinessAPI.cs

Auth: XApi header = LivelinessKey (appsettings)

Separate third-party (not NADRA, not Unikrew): Dedicated face-liveliness API authenticated with XApi header key from AppConfigurations:LivelinessKey in appsettings.json. Distinct from Unikrew's face comparison in section 9.

→ Request — VerifyLivelinessRequestEntity

```
{
  "Base64Image": "<base64-selfie>",
  "CNIC": "4210112345678"
}
```

← Response — VerifyLivelinessResponseEntity

```
{
  "StatusCode": "200",
  "StatusMessage": "Success",
  "ResponseObject": {
    "probability": "0.97",
    "score": "95",
    "quality": "good",
    "error": "",
    "error_code": ""
  }
}
```

12. Onboarding — Unikrew CNIC Document Validation

Used by: Account/ValidateDocumentUnikrew

Repo: AccountRepository.cs:57 → GetURL("ValidateDocumentUnikrew") — LIVE

Entity: ValidateDocumentUnikrewAPI.cs

Unikrew CNIC authenticity check: Validates whether the uploaded CNIC image is authentic and checks image quality (blur/brightness). Distinct from the face-comparison upload in section 9. userName/password injected from URL config; logged via SP usp_insert_cnic_document_validation_unikrew_logs.

→ Request — ValidateDocumentUnikrewRequestEntity

```
{
  "Base64Image": "<base64-cnic-image>",
  "userName": "<from url config>",
  "password": "<from url config>"
}
```

← Response — ValidateDocumentUnikrewResponseEntity

```
{
  "cnicDetails": {
    "side": "front",
    "type": "smart"
  },
  "details": {
    "isBlurry": false,
    "isTooBright": false,
    "isTooDark": false
  },
  "isAuthentic": true
}
```

13. Onboarding — Customer Detail

Used by: inside Account/Register flow (AccountService.cs:405)

Repo: AccountRepository.cs:272 → GetURL("CustomerDetail") — LIVE

Entity: CustomerDetailApi.cs

Core-banking profile fetch: Called during registration to pull the customer's profile by CNIC before creating the mobile-banking user. Standard core-banking envelope fields apply.

→ Request — CustomerDetailRequestEntity

```
{
  "strAppID": "", "strADC_TranCode": "", "strDelimiter": "",
  "strDate": "", "strTime": "", "strNetID": "",
  "intRefNum": "", "intSTAN": "", "strTranMsg": "",
  "strMerName": "", "strMCC": "", "strToken": "",
  "ClientIP": "", "DeviceIP": "", "Deviceversion": "", "DeviceType": "",
  "DeviceMEI": "", "ApplicationVersion": "", "UUID": "",
  "CustomerID": "111", "Latitude": "", "Longitude": "",
  "Channeltype": "", "CNIC": "4210112345678"
}
```

← Response — CustomerDetailResponseEntity

```
{
  "Response": {
    "CNIC": "4210112345678",
    "FULL_NAME": "ALI KHAN",
    "Email": "ali.khan@example.com",
    "ADDRESS": "House 123, Street 4, Karachi",
    "MOBILE_NUM": "03001234567",
    "D_Birth": "1990-01-01"
  },
  "Response_Code": "00",
  "Response_Desc": "Success"
}
```

14. Inquiry — Accounts by CNIC

Used by: Inquiry/CNIC

Repo: InquiryRepository.cs:103 → GetURL("CnicTitleFetch") — LIVE

Entity: CnicTitleFetchApi.cs

Core-banking (not NADRA): Returns all CASA accounts held against a CNIC. SOAP-style response shape (Res → GetAccountFromCNICItems → Item).

→ Request — CnicTitleFetchRequestEntity

```
{
  "strAppID": "", "strADC_TranCode": "", "strDelimiter": "",
  "strDate": "", "strTime": "", "strNetID": "",
  "intRefNum": "", "intSTAN": "", "strTranMsg": "",
  "strMerName": "", "strMCC": "", "strToken": "",
  "CNIC": "4210112345678",
  "R1": "", "R2": "", "R3": "", "R4": ""
}
```

← Response — CnicTitleFetchResponseEntity

```
{
  "Res": {
    "NS1": "",
    "GetAccountFromCNICItems": {
      "Item": {
        "Title": "ALI KHAN",
        "AccountNumber": "0345001234567"
      }
    },
    "Result": "Success"
  }
}
```

Endpoint ↔ Third-Party Call Summary

Endpoint	Request Entity	Response Entity	Repo Call Site	Status
Payment/IFT	IftPaymentRequestEntity	IftPaymentResponseEntity	FundsTransferRepository.cs:32	Mocked (service)
RAAST/Ibft	IftPaymentRequestEntity	IftPaymentResponseEntity	FundsTransferRepository.cs:32	Mocked (service)
Inquiry/IBFT	IbftPaymentRequestEntity	IbftPaymentResponseEntity	FundsTransferRepository.cs:64	Mocked (service)
Inquiry/IBFT_V1	IbftPaymentRequestEntity	IbftPaymentResponseEntity	FundsTransferRepository.cs:64	Mocked (service)
Inquiry/IFT	IftTitleFetchRequestEntity	IftTitleFetchResponseEntity	InquiryRepository.cs:142	Mocked (service)
Inquiry/Balance	BalanceInquiryRequestEntity	BalanceInquiryResponseEntity	InquiryRepository.cs:37	Mocked (service)
Account/Login	LoginRequestEntity	LoginResponseEntity	AccountRepository.cs:170	Real (SP-validated)
Account/Register	RegisterRequestEntity	RegisterResponseEntity	AccountRepository.cs:213	Real
Account/GetAccountList	AccountListRequestEntity	AccountListResponseEntity	AccountRepository.cs:110	Mocked
Account/UploadDocumentsUnikrew	UploadDocumentUnikrewRequestEntity	UploadDocumentUnikrewResponseEntity	AccountRepository.cs:82	Mocked
Account/VerifyFingers (NADRA)	VerifyFingerRequestEntity	VerifyFingerResponseEntity	AccountRepository.cs:13	Live (NADRA)
Account/VerifyLiveliness	VerifyLivelinessRequestEntity	VerifyLivelinessResponseEntity	AccountRepository.cs:40	Live
Account/ValidateDocumentUnikrew	ValidateDocumentUnikrewRequestEntity	ValidateDocumentUnikrewResponseEntity	AccountRepository.cs:57	Live
Account/Register (CustomerDetail step)	CustomerDetailRequestEntity	CustomerDetailResponseEntity	AccountRepository.cs:272	Live
Inquiry/CNIC	CnicTitleFetchRequestEntity	CnicTitleFetchResponseEntity	InquiryRepository.cs:103	Live